

Plugin Scenario -

The business requirement was to track how an asset depreciates over time using the declining balance (WDV) method within Dynamics 365 Field Service. The system needed to calculate depreciation annually based on the current asset value until it reaches the defined residual value or the end of its useful life.

The calculation followed:

Depreciation = Current Asset Value × Depreciation Rate

New Value = Current Value - Depreciation

The solution was implemented using Dynamics 365 Field Service (Dataverse) by extending the Customer Asset entity with custom fields such as Purchase Cost, Current Value, Depreciation Rate (%), Residual Value, and Last Depreciation Date. A Ribbon customization using Ribbon Workbench added a "Calculate Depreciation" button on the asset form, enabling users to trigger the calculation on demand. JavaScript was used to invoke a Custom Action, passing parameters like Asset ID and current values.

The core business logic was implemented in a C# Plugin registered on the Custom Action, ensuring centralized and controlled execution. The plugin calculated depreciation using the declining balance method by always applying the percentage on the latest current value. It included validations to ensure that the calculated value never dropped below the residual value, and if the computed value was less than or equal to the residual value, the plugin capped the value at the residual threshold and stopped further depreciation. The plugin then updated the asset record with the new depreciated value and maintained consistency of financial data. This approach ensured accurate calculation, enforcement of business rules at the server level, and reusability of the logic across different execution points such as manual trigger and future automation.

Javascript scenarios -

1. When a Case is created and a Contact is selected → system automatically fills related details like pricing, location, etc., ensuring data consistency and reducing manual effort; certain fields can be auto-filled and made read-only or hidden to avoid incorrect updates
2. When Incident Type is selected → corresponding Work Order Type is populated to standardize service processes; the field can be auto-filled and locked (disabled) to enforce correct classification
3. On button click actions → system validates conditions before executing, shows user confirmation dialogs, and prevents duplicate records; buttons are shown/hidden or enabled/disabled based on business rules and record state
4. During job booking → system displays available time slots and technicians in a structured dialog, allowing users to select appropriate resources; options can be dynamically enabled/disabled based on availability
4. Approve/Reject actions → only authorized users can take action based on their role or ownership; buttons are visible or enabled only for eligible users to enforce approval hierarchy
5. Customer Asset vs Agreement date validation → system ensures service dates fall within valid agreement periods, preventing invalid transactions; users are notified of errors and restricted from saving until corrected, with fields optionally locked

Common syntaxes used -

`Xrm.WebApi.retrieveMultipleRecords("entity", "?$filter=field eq value").then(success, error);` → fetch multiple records with filter

`Xrm.WebApi.createRecord("entity", data).then(success, error);` → create new record

`Xrm.Navigation.openConfirmDialog({ text: "Are you sure?" }).then(result => { if (result.confirmed) {} });` → show confirmation dialog

`Xrm.Navigation.openAlertDialog({ text: "Message" });` → show alert message

```
formContext.ui.refreshRibbon(); → refresh ribbon buttons
Xrm.WebApi.execute(request).then(response => response.json()); → execute
custom action / API
Xrm.Navigation.openWebResource("webresource.html", options, data); → open
HTML dialog
var obj = JSON.parse(jsonString); → convert string to object
window.postMessage(data, "*"); → send data between windows
var userId = Xrm.Utility.getGlobalContext().userSettings.userId; → get
logged-in user
Xrm.WebApi.retrieveRecord("entity", id, ")?$select=field").then(success,
error); → fetch single record
formContext.getControl("field").setNotification("Error"); → show validation
error
formContext.getControl("field").clearNotification(); → remove validation
error

var formContext = executionContext.getFormContext(); → get form context
formContext.getAttribute("field").getValue(); → read field value
formContext.getAttribute("field").setValue(value); → set field value
formContext.getControl("field").setVisible(true/false); → show/hide field
formContext.getAttribute("field").setRequiredLevel("required"); → make field
mandatory
formContext.getAttribute("field").addOnChange(functionName); → trigger logic
on change
formContext.getAttribute("field").fireOnChange(); → manually trigger change
formContext.getControl("field").setNotification("Error"); → set error message
formContext.getControl("field").clearNotification(); → clear error message
```